

Сканиране за уязвимости: колко реално се експлоатира?

Приоритизация чрез exploitability и reachability анализ

Лектор: Деница Танева

Senior Security Engineer | WiseBee

ВСИЧКИ НАХОДКИ

РЕАЛЕН РИСК

Сканерите намират всичко, но за какво реално имаме време?

~131

нови CVE на ден

средно през 2025 г. (~127-131/ден)

320 000+

CVE общо в базата

към декември 2025 г.

43 дни

Until Fixed

(от 32 дни година по-рано)

26%

Fixed
(KEV)

Същинският проблем не е откриването – а триажът.

Ако всяка „critical“ находка се третира еднакво, екипът изразходва ресурс по неща, които никой не атакува, докато реално опасното чака на опашката.

CVSS измерва тежест, не вероятност да те атакуват

CVSS – severity



Описва присъщите характеристики

вектор, нужен достъп, потенциален ефект



Статичен

обикновено не се променя след публикуване



Не отчита реалната активност

много 7.0+ CVE никога не се атакуват

EPSS – likelihood



Вероятност за експлоатация в следващите 30 дни

стойност от 0 до 1



Динамичен

обновява се ежедневно с нови данни за заплахи



Управляван от FIRST, безплатен

комплементарен на CVSS, не го заменя

Данни: само **~2,3%** от CVE с CVSS ≥ 7 реално са наблюдавани в опит за експлоатация. Тежест $\neq$ риск.

Колко от находките реално могат да се експлоатират?

~6%

от всички публикувани CVE

някога са били експлоатирани

<2%

експлоатирани „in the wild“

по данни на Kenna Security / Cyentia

0,004%

са в CISA KEV

1 414 от 312 011 CVE с потвърден случай на експлоатация

Изводът: огромната част от „червените“ находки в един отчет **никога няма да бъдат атакувани**. Въпросът е кои са важните - и тук влизат exploitability и reachability.



Когато CVE се експлоатира, прозорецът е тесен

28,3%

от експлоатациите – до 1 ден след разкриване
(VulnCheck, Q1 2025)

≈1 на 3

експлоатации настъпват в рамките на 24 часа –
фактически zero-day условия

+56%

ръст на атаките срещу уеб уязвимости спрямо
предходната година



DBIR 2026

**Експлоатацията на уязвимости
вече е причина #1 за пробив,**

изпреварвайки откраднатите идентификационни
данни.

Парадоксът на триажа: малко находки се експлоатират – но точно те се движат най-бързо. Затова сляпото подреждане по CVSS губи състезанието.

EPSS: вероятностен модел, обучен върху реални атаки

Exploit Prediction Scoring System (FIRST)

1 Обучение върху 1 100+ променливи

вендор, възраст, CWE (common weakness enumeration), CVSS, exploit код, мрежов трафик

2 „Ground truth“ от реални наблюдения

honeypot-и, IDS/IPS, доклади на изследователи - обновявани ежедневно

3 Изход: вероятност 0–1 за следващите 30 дни

0,96 ≈ 96% шанс за експлоатация в следващия месец

Две числа – не ги бъркайте

EPSS score

Абсолютен риск – самата вероятност.

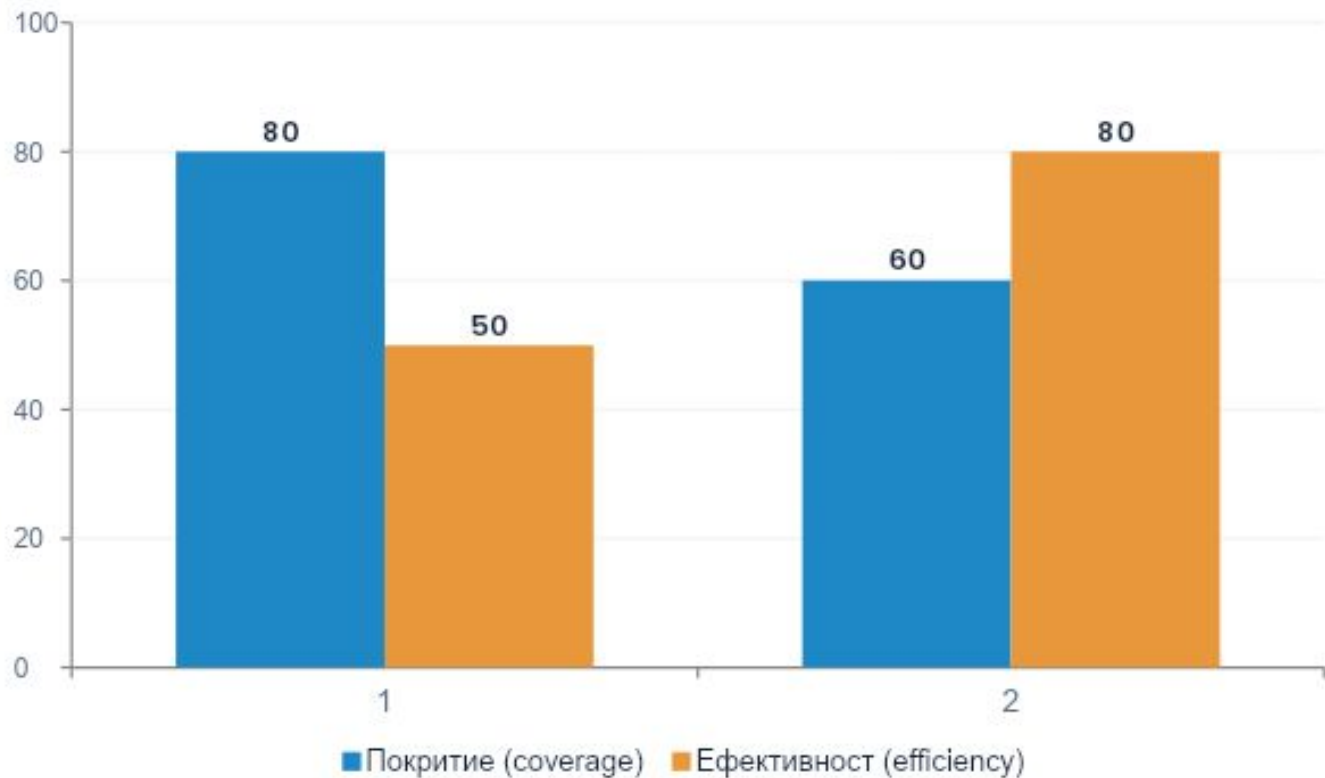
0,45 → 45% шанс за експлоатация

EPSS percentile

Относителен ранг спрямо всички CVE – не е вероятност.

percentile 98% → 98% от CVE са с по-нисък score

Покритие срещу ефективност: къде да сложиш чертата



Покритие

Какъв дял от реално експлоатираните CVE улавяш.
Нисък праг = ловиш почти всичко, но и много шум.

Ефективност

Какъв дял от закърпеното реално е било нужно.
Висок праг = почти няма излишна работа, но изпускаш част от находките.

Приоритизацията е “фуния” от филтри, не едно число



Reachability: присъствие $\neq$ изпълнява се

Reachability analysis определя дали уязвимият код реално се достига. Ако изпълнението никога не стига до уязвимата функция, тя е много по-малко вероятно да бъде експлоатирана.



Налично, но НЕдостижимо

Логваща библиотека има critical CVE, но уязвимата функция никога не се извиква в продукция.

= нисък приоритет; може да се маркира „Not Affected“



Достижимо от недоверен вход

DB драйвер със същия CVSS, но уязвимата функция е директно достижима от интернет-изложен API.

= висок приоритет; оправи първо това

Нива на анализ – от грубо към прецизно



Package-level

Само от манифеста (package.json, requirements.txt). Казва кои пакети НЕ са достижими, но не и кои реално са.

Груб; висок шум



Class-level

Грубо проследяване до клас. По-добре от нищо, но често имитира анализ.

Шум ↓ само ~20–40%



Function-level

Call graph до конкретната уязвима функция. Извиква ли я (пряко/косвено) кодът.

Шум ↓ до ~90%+



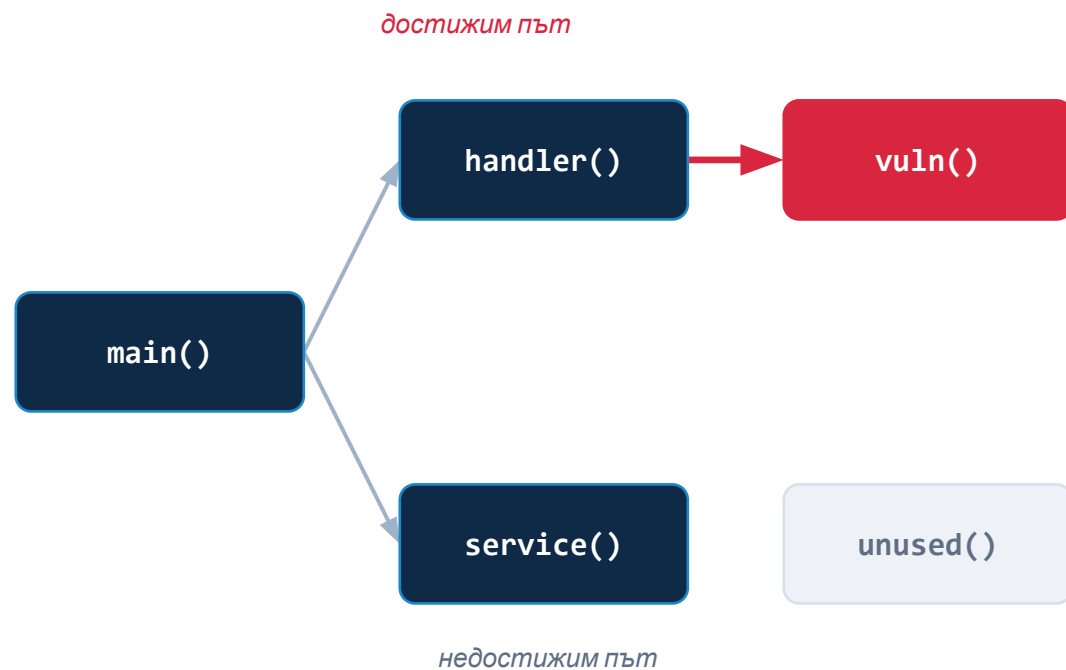
Dataflow

Разглежда най-задълбочено кодът. Най-висока увереност.

Най-прецизно

По-задълбочено = по-малко шум, но по-бавно и по-сложно. Class Hierarchy Analysis (CHA) е бързо, но неточно - приема, че всяко извикване може да удари всеки подклас.

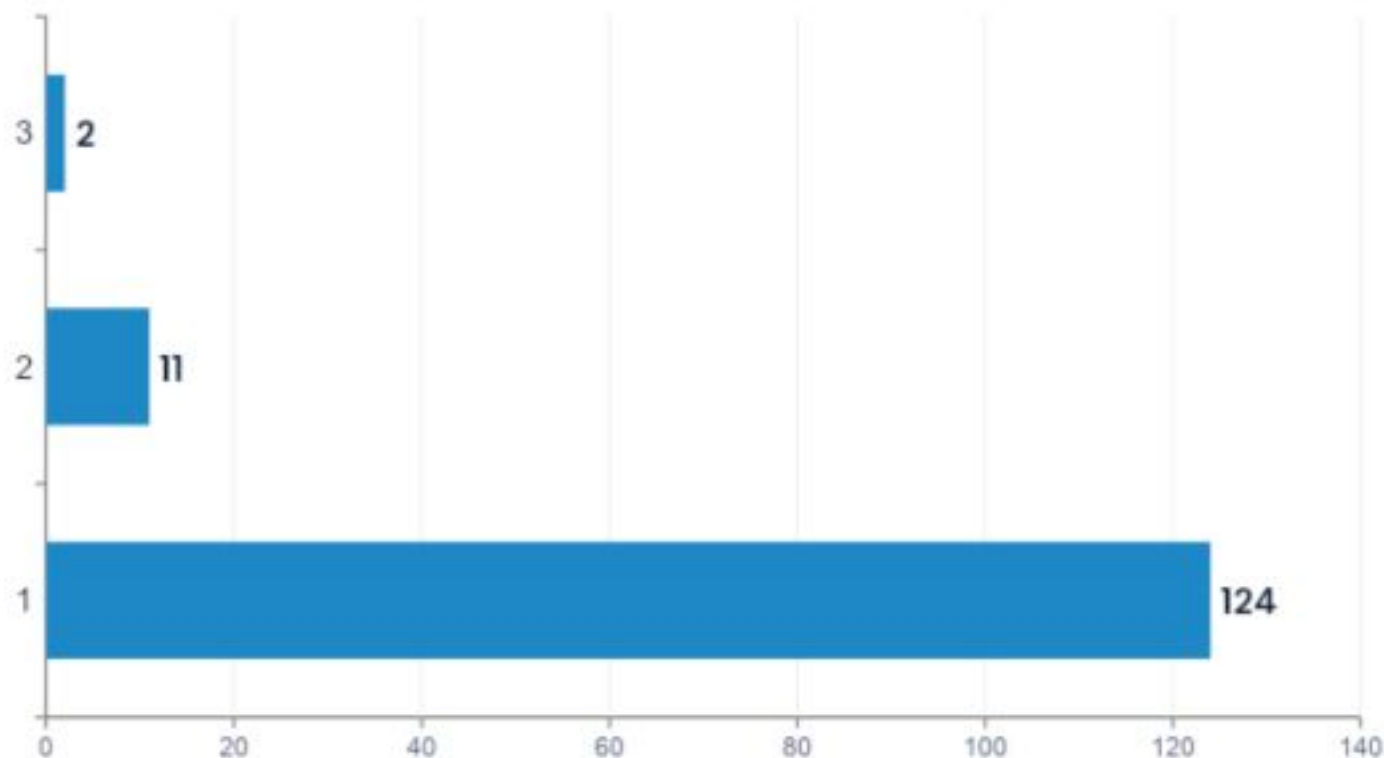
Call graph



Стъпки на статичния анализ

- 1 Парсва се кодът на 1-st party приложението (анализ и преобразуване в структуриран формат)
- 2 Гради се call graph от функциите и извикванията между тях
- 3 Уязвима функция в зависимостта се локализира
- 4 Търси се път - дали ще достигнем функцията?
- 5 Има път = reachable; няма = unreachable

От 124 находки към 2, които наистина значат



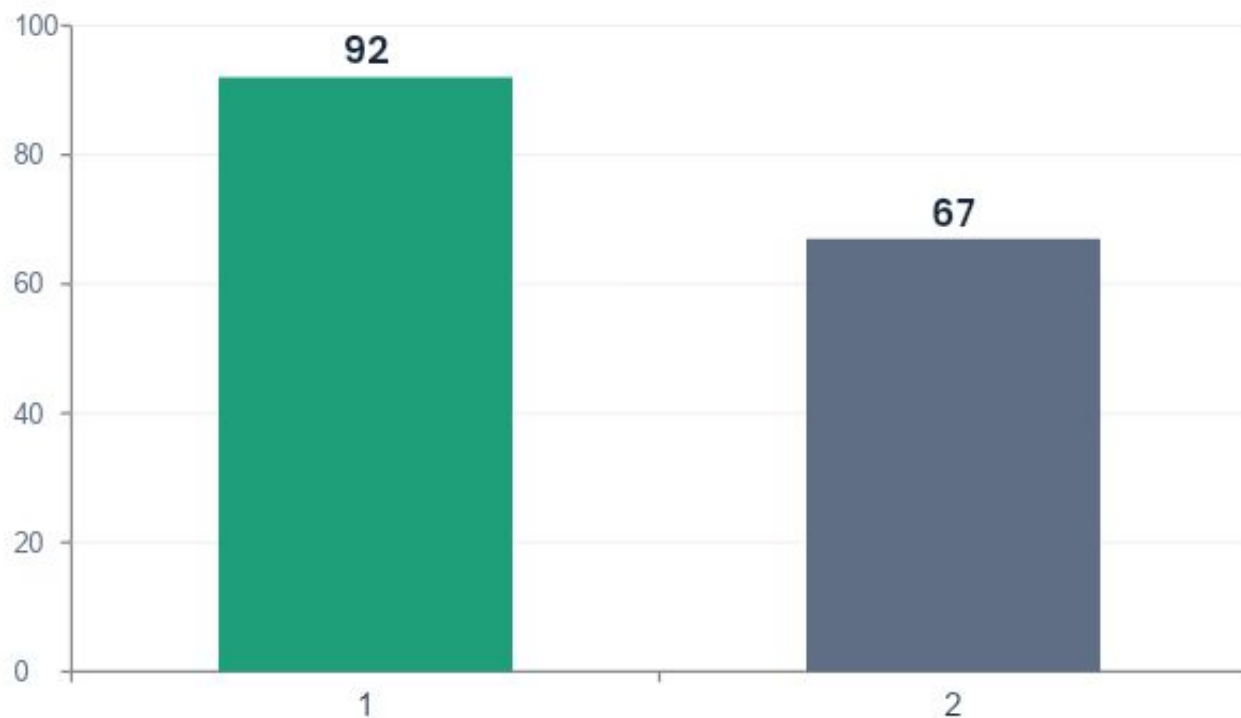
~91%

намаление само от function-level reachability
като филтър

В други среди числата са сходни:

до ~92% намаление на шума при function-level reachability; в някои среди 90%+ от находките се маркират като недостижими.

Достижимите находки реално се оправят по-често



Сълб 1 = достижими находки · Сълб 2 = останали находки (стойност = % от находките, които екипите оправят)

Защо това има значение



Reachability намалява алармите

сигналът отделя „технически наличното“ от „реално експлоатируемото“



По-добра диагностика на програмата

ако fix rate на критичните SAST е под 50%, проблемът най-често не е в откриването



Фокус върху правилните неща

ресурсът отива там, където атакуващият реално може да стигне

Reachability е силен сигнал – но не доказателство



Динамично извикване

извиквания през reflection, eval, dependency injection или сериализация може да не личат в call графа = false negatives



Грешна гранулярност

package- и class-level „reachability“ дава фасада на анализ; ниска точност и при двата вида грешки



Времето променя картината

недостижима днес функция може да стане достижима след рефакторинг или нова входна точка



One entry point, not a final result

reachability е едно от измеренията; комбинира се с EPSS, изложеност към интернет и чувствителност на данните

Reachability + VEX: документиран „Not Affected“

Когато даден CVE е доказано недостижим, той може да получи VEX статус „Not Affected“ - с проследима, одитна обосновка.



Reachable

Доказан път до уязвимата функция

Affected / Investigate



Not reachable

Няма път в call графа

Not Affected

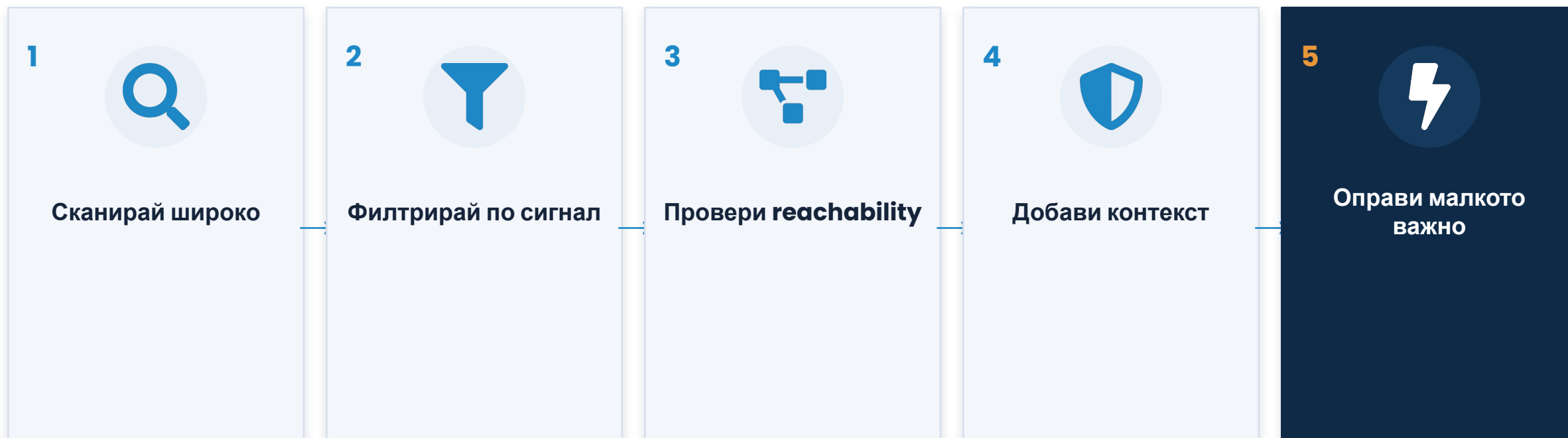


VEX документ

Машинно четим запис на решението

За одит

Практическа рамка за приоритизация



*Целта не е нулев брой находки. Целта е правилните находки да са горе в списъка - преди атакуващият да стигне до тях.
Юни 2026: новата CISA BOD 26-04 официално налага точно този подход - приоритизация по риск, не по тежест.*

Open source инструменти: от detected до exploitable

Три безплатни слоя превръщат стената от находки в подреден по риск план - а OpenCVE ги обединява в едно табло.



1 • KEV

CISA KEV · VulnCheck KEV



2 • EPSS

FIRST EPSS · CVE_Prioritizer



3 • Reachability

naabu · httpx · nuclei

Сканирай за инструментите и целия процес



Три неща, които да запомните

01

Малкото важно е малко

~6% от CVE някога се експлоатират; под 2% „in the wild“. Сканерът показва шум - твоята работа е сигналът.

02

Сигналът се наслоява

CVSS казва колко е тежко, EPSS - колко е вероятно, KEV - кое вече се атакува. Заедно, не поотделно.

03

Reachability намалява шума

Function-level / dataflow анализ маха до ~90% от находките като недостижими - и документира решението чрез VEX.

? Въпроси?

Благодаря!

ИЗТОЧНИЦИ

Cyentia Institute & FIRST - A Visual Exploration of Exploitation in the Wild (EPSS), 2024

Kenna Security & Cyentia - Prioritization to Prediction (P2P) серия

FIRST.org - EPSS модел и документация

Verizon - Data Breach Investigations Report (DBIR) 2026

VulnCheck - Exploitation Trends Q1 2025; CISA KEV; S2W анализ 2025

Semgrep - Reachability Analysis whitepaper (2025); Endor Labs; Apiiro; Hopper Security; Pixee

Statement of Non-Endorsement

The Open Worldwide Application Security Project (OWASP) is a 501(c)(3) worldwide not-for-profit charitable organization focused on improving the security of software. OWASP does not endorse or recommend any product or service. This allows our community to remain vendor neutral with the collective wisdom of the best minds in software security worldwide.

Споменатите инструменти са посочени само като примери за подходи; не представляват препоръка.